

XRPL Validator — Daily Engineering Report • 2026-09-01

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **43 commits** (xrpl-validator 42, xrpl-sdk 1)
- **1 engine finding fixed** (F51)
- 3 test / vector commits

Engine findings fixed

- **fix(ledger): scope F51's mid-floor and leg-B reprice to the in-exhausted slice** (79aeb9b , xrpl-validator) soak-98 caught the over-reach at #106674447 (58 clean then cascade): 2049BE47 buys exactly 1.0 RLUSD — an OUT-limited bridged pass, where rippled sizes by ceil_out and the mid rounds UP for pool legs too (its iteration outs sum to 1.0000000000000000 verbatim). Flooring every pool mid broke six of its nodes. Only the IN-EXHAUSTED slice floors the pool mid and reprices leg B off the whole drop; every other pass keeps ...

Engineering changes

- **fix(ledger): offer crossing consumes flowCross's grossed sendMax budget (findings 50-51)** (addcbc1 , xrpl-validator) rippled grosses sendMax ONCE at flowCross entry (multiplyRound roundUp, OfferCreate.cpp:354) and flow() consumes THAT budget: the in-limited final fill/slice takes the remaining GROSS verbatim and derives its net by division (limitStepIn mulRatio roundDown). Our walks tracked the NET budget and re-grossed per leg — one conversion too many, one ulp per crossing: the six-fix era's dominant byte-diff family (class-B ...
- **fix(ledger): equal-withdraw fraction rounds NEAREST in every mode (finding 54)** (2a3af82 , xrpl-validator) The F31/32 mode fork (withdrawAll ceils) was a phantom: #106629211's 'ceil uniquely required' was under-determined — both candidate fractions land byte-identical lines once the withdrawer add and pool subtract quantize at their own exponents. #106692584 B27127D6 (tfWithdrawAll, Jocker/XRP) does discriminate: ceil hands the withdrawer ...615 where mainnet's ...613 demands nearest — which also satisfies #105880685, ...
- **fix(ledger): AMMBid ported from AMMBid.cpp — the stub always succeeded (finding 52)** (5a55d3d , xrpl-validator) Preclaim: empty pool tecAMM_EMPTY; non-LP bidder tecAMM_INVALID_TOKENS; BidMin/BidMax above the bidder's LP or at/past the pool balance, or min > max → tecAMM_INVALID_TOKENS. applyBid: minSlotPrice = lptAMM × (tfec/100000) / 25; ammAuctionTimeSlot (24h window, 20 intervals); unowned/expired slot pays off minSlotPrice and burns it whole; an occupied slot prices at 1.05×purchase (interval 0) or decayed by 1 - ...
- **fix(ledger): CheckCreate holds the writer to the incremented reserve (finding 53)** (ebf7f62 , xrpl-validator) mPriorBalance < accountReserve(OwnerCount + 1) → tecINSUFFICIENT_RESERVE (CreateCheck.cpp); apply_common has already taken the fee here, so the prior balance is balance + fee. #106692368 FAA40191

(recurring: 106692434, 106692447 same spammer): a 12-object account at 5.63 XRP files checks — mainnet claims the fee alone; we wrote the Check plus both dir pages. Byte-exact tec vector pins it.

- **feat(node): encode-err receipts carry the error and the failing JSON** (3bb91cb , xrpl-validator) #106692562 B580FF73 (a rested FLR/USD offer) died unexplained: the bundle drill could not starve the same walk into the same write, and the receipt said only 'encode-err'. The arm now prints the encoder's error and OUR stored JSON before the reconcile erases them. Generator: book head fetch deepened to 32 offers per book.
- **fix(ledger): the maker pays ownerGives — out-issuer TransferRate on top of the fill (finding 55)** (3160205 , xrpl-validator) BookStep.cpp:808-830 verbatim: ownerGives = mulRatio(ofrAmt.out, ofrOutRate, roundUp=false), bounded by the owner's funds; a funding-limited fill sizes its out on funds/rate rounded DOWN, the maker is debited ownerGives (the raw funds VERBATIM when clamped), the taker receives the sized out, and the issuer destroys the difference. Parity when the owner or the strand destination is the issuer; zero/reat tests stay on ...
- **fix(ledger): IOU-in budgets arm the gross cap fully funded; rested amounts store 16-digit (findings 56-57)** (0d1ef3f , xrpl-validator) F56 (#106693639 8B7AE3, RLUSD → XRP tfSell|IoC): TakerGets equals the seller's entire RLUSD balance; rippled's per-iteration 16-digit remaining-in chain drains the line to EXACT ZERO across eight iterations while our exact me_sub chain kept 6e-7. sendMax IS the budget: the fully-funded IOU-in cap now arms (grossed when rated per F50, as-is unrated) and the exhausting fill takes the remainder verbatim — Finding 30's ...
- **fix(ledger): ownerGives is CROSSING-only — payment hops charge the out-fee at the next step** (4dfaf53 , xrpl-validator) soak-98 caught the second-site double charge at #106674462 (73 clean, 25-ledger cascade): the direct walk serves payment strands too, and a payment's following DirectStep already debits the maker's fee via rippleSend — charging ownerGives in the walk as well double-debited every rated payment fill. An offer crossing's strand ends at the taker, so the walk is the only site there (C3ECD3's maker drained to exact zero ...
- **fix(ledger): an out-side clamp supersedes in-exhaustion — the OUT-limited bridged slice is not a budget drain (finding 58)** (da0d8cd , xrpl-validator) F58 (#106674447 2049BE47, XAH → RLUSD tfIoC through the XRP bridge, buying EXACTLY 1.0 RLUSD; soak-98's 40-ledger cascade after F56): the bridged slice clamps its in FIRST — a raw pool offer above the remaining budget marks the slice IN-EXHAUSTED — and the out-cap / leg-B-funding clamp binds AFTERWARDS, re-deriving xrp and gets_in but never clearing the flag. Before F56 the flag could only arm behind a TransferRate, ...
- **feat(scripts): fetch_tx_bundle rebuilds same-ledger stale pre-images from the earlier toucher's binary meta** (424dc8a , xrpl-validator) A key touched by an EARLIER tx in the same ledger has a parent-ledger image that is not this tx's pre-image (#106674447: tx#5 credited the leg-B maker line 10 RLUSD before the crossing read it — the vector had to be hand-patched). The bundle now rebuilds such keys byte-exactly from the last earlier toucher's binary meta: ModifiedNode.FinalFields + LedgerEntryType (0x11) + the thread stamp PreviousTxnLgrSeq (0x25) / ...
- **fix(scripts): fetch_tx_bundle — replay directory-page entries, derive pre-images forward from this tx's own meta** (2d76a70 , xrpl-validator) The previous commit rebuilt a same-ledger-stale key from the earlier toucher's FinalFields — silently wrong for DirectoryNodes: sfIndexes is SMD_Never, so no meta carries a page's entries and the rebuilt page lost them. Pages are now REPLAYED: every object an earlier tx created/deleted names its pages (OwnerNode/BookNode/LowNode/HighNode/...), owner pages take sorted inserts and book pages appends (rippled dirInsert ...
- **fix(ledger): the payment driver runs on its remainders, not a 32-round cap (finding 59)** (761c148 , xrpl-validator) rippled's flow() loops while (remainingOut > 0 && *remainingIn > 0) and only its safety bound

ends it — the 1000th entry is telFAILED_PROCESSING (StrandFlow.h:606-657), a local failure no validated ledger carries. Ours was capped at 32 rounds: the old multi-strand interleave cap, kept for lone strands when every strand started re-entering (4566c4e), so a lone strand could take at most 32 fills-or-AMM-slices.

- **fix(ledger): SetRegularKey spends the password on the calculated base fee, and accepts the free form (finding 60)** (c8c11f3 , xrpl-validator) if (!minimumFee(app, ctx_.baseFee, fees, flags)) sle->setFlag(lsfPasswordSpent) (SetRegularKey.cpp:71): the flag follows the CALCULATED base fee — zero iff the tx is signed with the sender's master key and the flag is still clear (calculateBaseFee, SetRegularKey.cpp:29-49) — not the fee the sender chose to pay. We never set it, and preflight refused Fee: 0 outright although that is the canonical free form.
- **fix(ledger): equalWithdrawTokens' fraction carries divide()'s legacy +5 into Number's nearest canonicalisation (finding 61)** (7b85121 , xrpl-validator) rippled's STAmount divide(num, den, issue) (STAmount.cpp) hands the constructor the TRUNCATED muldiv(num, 10^17, den) quotient — 17 or 18 digits — plus the legacy half-up bias of 5. Under fixUniversalNumber canonicalize() is *this = iou():Number normalises the mantissa to 16 digits under the ambient to_nearest mode, so the +5 that once meant "round half up, then truncate" now rounds the quotient UP whenever ...
- **fix(ledger): AMMWithdraw snaps the sole LP's LPTokenBalance to its trust line before sizing (finding 63)** (30b7e65 , xrpl-validator) fixAMMv1_1 verifyAndAdjustLPTokenBalance (AMMUtils.cpp), run by AMMWithdraw::applyGuts before anything is sized: the last LP's trust line and the AMM's LPTokenBalance drift apart by rounding dust over the pool's life, so when the withdrawer is the ONLY liquidity provider — no other account holds an LPToken line in the AMM account's owner directory (isOnlyLiquidityProvider) — the object's LPTokenBalance is set to ...
- **fix(ledger): a funds-bound bridge leg B floors the XRP mid-leg — rippled's limitOut(roundUp=false) (finding 62)** (c55e13c , xrpl-validator) rippled's BookStep::forEachOffer sizes an offer whose OWNER FUNDS are the binding limit with limitOut(ofrAmt, stpAmt.out, /*roundUp*/ false) — "we can prevent order book blocking by (strictly) rounding down the ceil_out() result" (fixReducedOffersV1, BookStep.cpp:822-829) — while the remaining-out clamp (limitStepOut, :705-708) keeps roundUp=true. Our bridged crossing derived the mid-leg XRP the same way for both ...
- **perf(node): ws-sync lag — cap the FFI's empty RPC consults, hydrate the shadow off the sync loop, stop rebuilding the peer TLS connector** (60003e6 , xrpl-validator) Measured on the validator host on 2026-09-01 (memory project_validator_wssync_lag_2026_09_01): the ws-sync loop serializes fetch → FFI verify → native shadow → write per ledger and had no headroom over the ~3.8 s ledger cadence, so the backlog each launch's hydrate left (~65 ledgers) never drained and sat at 100-150.
- **fix(ledger): a first self-mint's FirstNFTokenSequence is the tx's sequence, not the bumped root's (finding 64)** (98a69a0 , xrpl-validator) NFTokenMint.cpp:263-275 (fixNFTokenRemint): when the issuer root has no sfFirstNFTokenSequence yet, it becomes the root's Sequence as-is only when the tx names an Issuer (an authorized minter — the issuer's Sequence was never consumed) or spends a Ticket; a plain self-mint takes acctSeq - 1, because the minter's own root has already been bumped past the tx's Sequence when doApply runs. We took the root's Sequence ...
- **fix(ledger): AccountSet honours the transaction-level tf flags, and RequireAuth refuses an owning account (finding 65)** (049b6af , xrpl-validator) SetAccount.cpp:326-336: bSetRequireDest = (uTxFlags & tfRequireDestTag) || uSetFlag == asfRequireDest, likewise RequireAuth and DisallowXRP, with tfOptionalDestTag / tfOptionalAuth / tfAllowXRP as the clears — the transaction-level flags are the old spelling of the same switches and rippled applies both (doApply :45-84). We only read SetFlag/ClearFlag.

- **fix(ledger): AMMDeposit and AMMWithdraw size at the auction-slot holder's discounted fee (finding 66)** (3efa4a3 , xrpl-validator) AMMDeposit.cpp:393 and AMMWithdraw.cpp:322 size with getTradingFee(view, ammSle, account) — the pool's TradingFee, or the AuctionSlot's DiscountedFee while the slot is unexpired and the account holds it or is one of its AuthAccounts (AMMUtils.cpp:179-205). The swap path already resolved that (amm_swap::discover); the four deposit/withdraw sizing sites read the raw TradingFee. The slot logic is now one helper, ...
- **feat(harness): bundles carry the ledger's TransactionResult and probe_bundle asserts on it (6084443 , xrpl-validator)** A tec specimen — the fee-only verdict rippled recorded — was unpinable: the probe asserted tesSUCCESS for every bundle. fetch_tx_bundle.py now stores metaData.TransactionResult as "result" and probe_bundle.rs asserts the engine's TER against it (tesSUCCESS when absent, so every older bundle is unchanged). First specimens: #106698812 5D50FA86 (tecKILLED, ours tecUNFUNDED_OFFER), #106699133 6B6670FD (tecAMM_FAILED, ...
- **fix(ledger): OfferCreate preclaim runs checkAcceptAsset on TakerPays — tecNO_ISSUER / tecNO_LINE / tecNO_AUTH (finding 67)** (90204ee , xrpl-validator) CreateOffer::preclaim (:63) calls checkAcceptAsset (:236-300) on the TakerPays issue after the expiry check: the issuer must exist (tecNO_ISSUER); under the issuer's lsfRequireAuth the taker must hold a line (tecNO_LINE) carrying the ISSUER-side auth bit — lsfLowAuth when the taker is the high account, lsfHighAuth otherwise (tecNO_AUTH); the issuer's own issuance always passes. We had no such check and crossed the ...
- **fix(ledger): one-asset withdraws honour Amount as a floor — tecAMM_FAILED under it (finding 68)** (6cd21f1 , xrpl-validator) AMMWithdraw::singleWithdrawTokens (AMMWithdraw.cpp:985-1016), the path of both tfOneAssetWithdrawAll (the LP's whole position) and tfOneAssetLPToken (an explicit LPTokenIn): the asset out is ammAssetOut(balance, lptAMMBalance, tokensAdj, tfee) and the withdrawal proceeds only if `amount == 0 || amountWithdraw >= amount` — otherwise tecAMM_FAILED, fee only. We treated Amount as the side's name and paid whatever the ...
- **fix(ledger): issuer-side NoRipple refuses the crossing book step (finding 69)** (effa17e , xrpl-validator) rippled BookStep::check (BookStep.cpp:1385-1401) reads the trust line between the previous direct step's source (the taker) and the book's in-issuer and fails the strand with terNO_RIPPLE when the issuer's side carries NoRipple. toStrands then has no strand, flowCross swallows the failure, and the offer rests whole (IoC/FoK: tecKILLED) without any offer being stepped.
- **fix(ledger): AccountSet clears a field on an empty or zero value (finding 70)** (7cf21ab , xrpl-validator) rippled SetAccount::doApply makes the field absent for an empty Domain or MessageKey blob, a zero EmailHash or WalletLocator, a TransferRate of 0 or QUALITY_ONE and a TickSize of 0 or 15; we copied the tx value across and filed an empty VL field.
- **fix(ledger): AMMVote fee rounds to nearest on the exact fraction (finding 71)** (ea74655 , xrpl-validator) rippled converts the weighted average with Number::operator rep(), which rounds up when the discarded fraction exceeds one half and to even only on an exact tie. We floored q*10 and read a trailing 5 as the tie test, so any fraction in [0.5, 0.6) rounded to even.
- **fix(core): accept rippled's ISO currency character set (finding 72)** (4dde5b5 , xrpl-sdk) rippled's detail::isoCharSet (UIntTypes.cpp:39-43) admits letters, digits and the symbols <>(){}[]!@#\$\$%^&* in a three-letter code, and never renders the system currency's ISO spelling as ISO. from_ascii rejected the symbols, so the validator could not re-encode a live W&B trust line (#106699694), and from_bytes would have rendered such a code as hex where rippled renders the ISO string.

- **fix(ledger): SignerList files the Owner keylet field (finding 73)** (68f1840 , xrpl-validator)
fixIncludeKeyletFields has rippled set sfOwner on the SignerList (SetSignerList.cpp:428-431) for a fresh list and a replacement alike. We never wrote it: #106699661 55F46ED0's one-entry list is 118 bytes on mainnet, 96 in ours — exactly the 22-byte Owner field. Pinned as signerlist_owner_106699661 (red at HEAD~, green here).
- **fix(ledger): nearest-drop conversion rounds exact ties to even (finding 74)** (25f69e7 , xrpl-validator)
XRPAmount{Number} converts through Number::operator rep(): above half rounds up, an exact half rounds to even. to_amount's Rnd::Near read the last digit of floor(x*10) and rounded every tie up, so 2.5 drops became 3 where rippled takes 2. Same family as finding 71; unit-tested (2.5 -> 2, 3.5 -> 4, the #105843839 limitOut drop unchanged).
- **fix(ledger): XRP fills feed the gross-spent cap of the tail pool slice (finding 75)** (35123ad , xrpl-validator) An XRP TakerGets fill is debited on the spot but was never added to in_gross_spent, so the tail AMM slice sized its verbatim debit from a stale cap (cap - in_gross_spent). #106700231 4CEDDA54 (funds-limited tfSell|tfIoC, XRP for RLUSD): slice 41,089,108 + fills 5,966,854 + 546,200 left a 5,181,445-drop tail; the tail asked for 11,694,499, the oversized debit was dropped, and the taker kept 5,181,445 drops the pool ...
- **fix(ledger): pool-vs-tip choice on the tip's raw filed quality (finding 76)** (90a7cfa , xrpl-validator) rippled's tryAMM(offers.tip().quality()) hands getOffer the tip AS FILED; the spot must beat it, and in multi-path the fib slice must beat it too (AMMLiquidity.cpp:78-92, BookStep.cpp:838-869) — ownerPaysTransferFee only shows at execution, never in the ranking. Our bridge discounted the tip by the output issuer's transfer rate before comparing, so a pool slice could outrank a better book tip. #106701467 DAE80780: ...
- **fix(ledger): the bridge controller's direct-pool slice pays the in-issuer's rate (finding 78)** (6a00cfb , xrpl-validator) rippled's BookStep grosses every step's in by trIn (BookStep.cpp:734-739); the direct walk's amm_turn sites already pass the rate, but cross_bridged handed consume/consume_fib None for the direct strand's pool slice, so the taker was debited NET and the slice never joined the gross-spend cap. #106701467 DAE80780: 1.257009702543272 FLR debited where mainnet takes 1.260780731650903. Pinned with finding 76 as ...
- **fix(ledger): a pool alone anchors a bridge leg with no book (finding 77)** (eeb2e89 , xrpl-validator) forEachOffer falls to tryAMM(std::nullopt) on an empty book and the single-path getOffer emits maxOffer, whose quality is the feeless spot (BookStep.cpp:866-869). Our single-path admission required a book tip on the leg, so a pool-only leg was never admitted. The 07-30 rule (413368a, 'carries nothing') had the right outcome on #105813899 44E799C6 for the wrong reason: rippled's trace admits and sizes the leg (in ...
- **fix(ledger): the resting remainder is re-derived from the offer's rate (finding 79)** (bd7f32e , xrpl-validator) flowCross (CreateOffer.cpp:439-467) rests one exact side and derives the other from the offer's own rate: buy keeps out and takes in = mulRound(out, rate, up); sell keeps in and takes out = divRound(in, rate, up), rate = TakerGets/TakerPays. We rested the walk's net remainder, one ULP short on #106701372 8BFFFACE. New helpers div_round16_up / div_round_drops_up mirror STAmount divRound. Pinned with finding 77 as ...
- **fix(ledger): an XRP leg never takes the IOU line path in move_leg_gross (finding 75b)** (0f2909d , xrpl-validator) XRP carries no transfer rate, so gross == net by construction; a caller handing a different gross sent the leg through line_adjust, which fabricated a RippleState against the zero account plus its owner directory — the phantom owner_dir(zero) A317582B seen in #106700231, #106701366 and #106701473 alongside finding 75. Net now always moves through the roots; unit-tested.
- **feat(harness): bundles carry pair pools, top-of-book, in-ledger creations and maker lines** (8f19ebf , xrpl-validator) fetch_tx_bundle.py now fetches: the AMM objects of the direct pair and both XRP-bridge pairs with their roots, owner dirs and lines (a pool fill never names the AMM in meta); the top of the direct and bridge books

via book_offers (an untouched tip keeps rippled's bridge strand active); objects and quality pages created earlier in the same ledger, rebuilt from the touchers (NewFields/FinalFields; page entries ...

- **fix(ledger): the sell remainder rounds down under fixReducedOffersV1 (finding 79b)** (760e8f1 , xrpl-validator) flowCross's tfSell arm rests out = divRoundStrict(in, rate, out.issue, roundUp = false) once fixReducedOffersV1 is on (CreateOffer.cpp:479-492); the up-rounding divRound is the pre-amendment arm, and the rate is getRate's legacy divide (17-digit truncate, +5, nearest) as rate_of_me encodes it. Finding 79's first cut rounded the sell side up and broke eight HIST60 ledgers (#106644297-306). #106644297 2B6E3D5A: ...

Tests and campaigns

- **test(ledger): create_account_at_exact_reserve expects the DeletableAccounts starting sequence** (03e5d6c , xrpl-validator)
- **test(node): PROBE_BUNDLE env-gated bundle prober** (0644cc6 , xrpl-validator)
- **test(node): pin a TrustSet creating a symbol-currency line (finding 72)** (f01956e , xrpl-validator)

Docs and operations

- **chore(ledger): clear the warning debt adjacent to the offer work** (002baee , xrpl-validator)
- **style(ledger): fold the F62 funds-bound flag into the mid-leg rescale condition (clippy nonminimal_bool)** (a18a466 , xrpl-validator)

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Bonded Checkpoint Light-Client Bridge Proofs** (interop) Add a new owner-reserved ledger object ltXCHAIN_CHECKPOINT (keylet::xChainCheckpoint(bridge, seq)) that a bridge's witness quorum posts periodically v
- **Segregated Sub-Accounts With Clawback-Gated Withdrawals** (custody) Add a new owner-reserved ledger object ltSUBACCOUNT (keylet::subAccount(parent, subSeq)), created by a SubAccountSet transaction from a parent Account
- **Sponsor-Funded Ticket Batches for Onboarding** (gaming) Add an sfSponsor field and tfSponsored flag to TicketCreate, letting a third-party sponsoring account — authenticated via the same prepay authorizatio

Appendix — commits

Time	Repo	Commit	Subject
09:47	xrpl-validator	addcbc1	fix(ledger): offer crossing consumes flowCross's grossed sendMax budget (findings 50-51)

Time	Repo	Commit	Subject
09:47	xrpl-validator	002baee	chore(ledger): clear the warning debt adjacent to the offer work
09:47	xrpl-validator	03e5d6c	test(ledger): create_account_at_exact_reserve expects the DeletableAccounts starting sequence
10:46	xrpl-validator	0644cc6	test(node): PROBE_BUNDLE env-gated bundle prober
10:46	xrpl-validator	79aeb9b	fix(ledger): scope F51's mid-floor and leg-B reprice to the in-exhausted slice
12:34	xrpl-validator	2a3af82	fix(ledger): equal-withdraw fraction rounds NEAREST in every mode (finding 54)
12:34	xrpl-validator	5a55d3d	fix(ledger): AMMBid ported from AMMBid.cpp — the stub always succeeded (finding 52)
12:34	xrpl-validator	ebf7f62	fix(ledger): CheckCreate holds the writer to the incremented reserve (finding 53)
12:41	xrpl-validator	3bb91cb	feat(node): encode-err receipts carry the error and the failing JSON
13:43	xrpl-validator	3160205	fix(ledger): the maker pays ownerGives — out-issuer TransferRate on top of the fill (finding 55)
14:09	xrpl-validator	0d1ef3f	fix(ledger): IOU-in budgets arm the gross cap fully funded; rested amounts store 16-digit (findings 56-57)
14:16	xrpl-validator	4dfaf53	fix(ledger): ownerGives is CROSSING-only — payment hops charge the out-fee at the next step
15:23	xrpl-validator	da0d8cd	fix(ledger): an out-side clamp supersedes in-exhaustion — the OUT-limited bridged slice is not a budget drain (finding 58)
15:51	xrpl-validator	424dc8a	feat(scripts): fetch_tx_bundle rebuilds same-ledger stale pre-images from the earlier toucher's binary meta
16:10	xrpl-validator	2d76a70	fix(scripts): fetch_tx_bundle — replay directory-page entries, derive pre-images forward from this tx's own meta
16:29	xrpl-validator	761c148	fix(ledger): the payment driver runs on its remainders, not a 32-round cap (finding 59)
16:31	xrpl-validator	c8c11f3	fix(ledger): SetRegularKey spends the password on the calculated base fee, and accepts the free form (finding 60)
17:14	xrpl-validator	7b85121	fix(ledger): equalWithdrawTokens' fraction carries divide()'s legacy +5 into Number's nearest canonicalisation (finding 61)
17:14	xrpl-validator	30b7e65	fix(ledger): AMMWithdraw snaps the sole LP's LPTokenBalance to its trust line before sizing (finding 63)
17:27	xrpl-validator	c55e13c	fix(ledger): a funds-bound bridge leg B floors the XRP mid-leg — rippled's limitOut(roundUp=false) (finding 62)
18:11	xrpl-validator	60003e6	perf(node): ws-sync lag — cap the FFI's empty RPC consults, hydrate the shadow off the sync loop, stop rebuilding the peer TLS connector
18:17	xrpl-validator	98a69a0	fix(ledger): a first self-mint's FirstNFTTokenSequence is the tx's sequence, not the bumped root's (finding 64)
18:34	xrpl-validator	049b6af	fix(ledger): AccountSet honours the transaction-level tf flags, and RequireAuth refuses an owning account (finding 65)
18:45	xrpl-validator	3efa4a3	fix(ledger): AMMDeposit and AMMWithdraw size at the auction-slot holder's discounted fee (finding 66)
19:36	xrpl-validator	a18a466	style(ledger): fold the F62 funds-bound flag into the mid-leg rescale condition (clippy nonminimal_bool)
19:42	xrpl-validator	6084443	feat(harness): bundles carry the ledger's TransactionResult and probe_bundle asserts on it

Time	Repo	Commit	Subject
19:46	xrpl-validator	90204ee	fix(ledger): OfferCreate preclaim runs checkAcceptAsset on TakerPays — tecNO_ISSUER / tecNO_LINE / tecNO_AUTH (finding 67)
19:49	xrpl-validator	6cd21f1	fix(ledger): one-asset withdraws honour Amount as a floor — tecAMM_FAILED under it (finding 68)
20:12	xrpl-validator	effa17e	fix(ledger): issuer-side NoRipple refuses the crossing book step (finding 69)
20:14	xrpl-validator	7cf21ab	fix(ledger): AccountSet clears a field on an empty or zero value (finding 70)
20:21	xrpl-validator	ea74655	fix(ledger): AMMVote fee rounds to nearest on the exact fraction (finding 71)
20:26	xrpl-sdk	4dde5b5	fix(core): accept rippled's ISO currency character set (finding 72)
20:34	xrpl-validator	f01956e	test(node): pin a TrustSet creating a symbol-currency line (finding 72)
21:11	xrpl-validator	68f1840	fix(ledger): SignerList files the Owner keylet field (finding 73)
21:11	xrpl-validator	25f69e7	fix(ledger): nearest-drop conversion rounds exact ties to even (finding 74)
21:17	xrpl-validator	35123ad	fix(ledger): XRP fills feed the gross-spent cap of the tail pool slice (finding 75)
22:38	xrpl-validator	90a7cfa	fix(ledger): pool-vs-tip choice on the tip's raw filed quality (finding 76)
22:38	xrpl-validator	6a00cfb	fix(ledger): the bridge controller's direct-pool slice pays the in-issuer's rate (finding 78)
22:39	xrpl-validator	eeb2e89	fix(ledger): a pool alone anchors a bridge leg with no book (finding 77)
22:39	xrpl-validator	bd7f32e	fix(ledger): the resting remainder is re-derived from the offer's rate (finding 79)
22:40	xrpl-validator	0f2909d	fix(ledger): an XRP leg never takes the IOU line path in move_leg_gross (finding 75b)
22:40	xrpl-validator	8f19ebf	feat(harness): bundles carry pair pools, top-of-book, in-ledger creations and maker lines
23:54	xrpl-validator	760e8f1	fix(ledger): the sell remainder rounds down under fixReducedOffersV1 (finding 79b)